

SESSION 6

Decision Trees

The Intuitive Power of Nested Decision Rules

What is a Decision Tree?

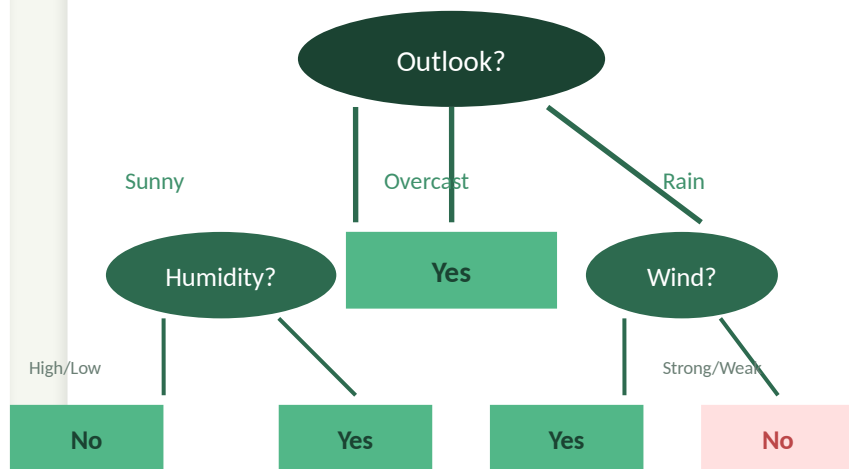
A supervised ML algorithm that makes predictions by learning nested decision rules from data.

Used for Classification AND Regression

- Root Node — First decision (top of tree)
- Decision Nodes — Internal nodes testing a feature
- Leaf Nodes — Final predictions (terminal)
- Branches — Outcomes of each test (Yes/No)

How it predicts: Follow the path from root to leaf, testing conditions at each node.

Example: Should I Play Tennis?



Trees are popular because they are easy to interpret, require minimal preprocessing, and handle both numerical and categorical features.

Building Intuition: The Tree Farm Problem

Scenario

You have a plot with Apple, Cherry, and Oak trees. Given only trunk Diameter and Height, predict the tree type. Each split divides the feature space with an axis-aligned boundary — creating rectangular regions.

1. First Split

Diameter ≤ 0.45
separates Oaks

2. Second Split

Height ≤ 4.88
isolates Cherries

3. More Splits

Further partitions
for Apple vs Cherry

4. Stop!

Too deep = overfitting!
Stop when good enough

The tree creates axis-aligned decision boundaries. This is both its strength (simple) and weakness (can't do diagonal splits).

Where to Split? Understanding Entropy

Entropy: Measuring Impurity

Entropy measures how "mixed" a set of samples is. Higher entropy = more uncertainty = more impure.

$$H = - \sum p_i \cdot \log_2(p_i)$$

- $H = 0 \rightarrow$ Pure node (one class only)
- $H = 1.0 \rightarrow$ Max impurity (binary, 50/50)
- $H = \log_2(k) \rightarrow$ Max for k classes

Entropy Examples (Binary Class)

Pure Node

10 cats, 0 dogs

$$H = 0.000$$

Slightly Mixed

9 cats, 1 dog

$$H = 0.469$$

Very Mixed

6 cats, 4 dogs

$$H = 0.971$$

Maximum

5 cats, 5 dogs

$$H = 1.000$$

Goal: Each split should REDUCE entropy — moving from impure parent nodes to purer child nodes.

Entropy: Worked Example

Dataset: 10 animals — 6 Cats and 4 Dogs

$$p(\text{Cat}) = 6/10 = 0.6$$

$$p(\text{Dog}) = 4/10 = 0.4$$

Step-by-Step

$$H = - [p(\text{Cat}) \cdot \log_2(p(\text{Cat})) + p(\text{Dog}) \cdot \log_2(p(\text{Dog}))]$$

$$H = - [0.6 \times (-0.737) + 0.4 \times (-1.322)]$$

$$H = - [-0.442 + (-0.529)]$$

$$H = - (-0.971) = 0.971$$

$$H = 0.971$$

(Close to max = 1)

For 3 classes (Apple/Cherry/Oak), max entropy = $\log_2(3) \approx 1.585$. For binary, max = $\log_2(2) = 1.0$.

Information Gain: Choosing the Best Split

Information Gain measures how much a split REDUCES entropy. We compare the parent entropy to the weighted child entropies. The split with the HIGHEST IG wins!

$$\text{IG} = H(\text{parent}) - (1/N) \times \sum N_{\text{child}} \times H(\text{child})$$

$H(\text{parent})$ = entropy before | $H(\text{child})$ = entropy of each child | N_{child}/N = proportion

ID3 Algorithm

1. Calculate entropy for every possible split (each feature $\times$ each threshold value)
2. Compute Information Gain for each candidate split
3. Choose the split with maximum IG $\rightarrow$ create decision node
4. Recurse on each child node until stopping criteria are met

The ID3 algorithm is GREEDY — it picks the locally best split at each step without backtracking.

Information Gain: Worked Example

Parent: 6 Cats, 4 Dogs ($H = 0.971$)

Split on: "Has Whiskers?"

Left (Yes): 5 Cats, 1 Dog

Right (No): 1 Cat, 3 Dogs

Child Entropies

$$\begin{aligned} H(\text{left}) &= -(5/6)\log_2(5/6) - (1/6)\log_2(1/6) \\ &= 0.650 \end{aligned}$$

$$\begin{aligned} H(\text{right}) &= -(1/4)\log_2(1/4) - (3/4)\log_2(3/4) \\ &= 0.811 \end{aligned}$$

Computing Information Gain

$$\begin{aligned} IG &= 0.971 - [(6/10) \times 0.650 + (4/10) \times 0.811] \\ IG &= 0.971 - [0.390 + 0.324] = 0.971 - 0.714 \\ \mathbf{IG} &= \mathbf{0.257} \end{aligned}$$

IG = 0.257

Good split!

We compute IG for EVERY possible feature $\times$ threshold combination. The highest IG becomes the next decision node.

Making a Prediction: Trace Through the Tree

New Data Point: Diameter = 0.30, Height = 6.5 — What type of tree?

Follow the path from root to leaf, testing each condition:

Step 1

Diameter ≤ 0.45 ?

$0.30 \leq 0.45 \rightarrow \text{YES}$

Go LEFT

Step 2

Height ≤ 4.88 ?

$6.5 \leq 4.88 \rightarrow \text{NO}$

Go RIGHT

Step 3

Diameter ≤ 0.32 ?

$0.30 \leq 0.32 \rightarrow \text{YES}$

Go LEFT

Step 4

Leaf Node reached!

Predict: Apple

This is the tree from the farmer example (Apple/Cherry/Oak). Each new sample follows ONE path down the tree.

Prediction = following a single path. That's why Decision Trees are so interpretable — you can trace every prediction!

Gini Impurity: An Alternative to Entropy

Gini Impurity

Probability of misclassifying a randomly chosen sample.

$$\text{Gini} = 1 - \sum p_i^2$$

Gini = 0 → Pure | Gini = 0.5 → Max (binary) | Faster: no logs!

Entropy vs Gini

	Entropy	Gini
Formula	$-\sum p_i \log_2(p_i)$	$1 - \sum p_i^2$
Range (bin.)	0 to 1	0 to 0.5
Speed	Slower (log)	Faster
Default?	No	Yes ✓

Quick Example: 6 Cats, 4 Dogs

$\text{Gini} = 1 - (0.6^2 + 0.4^2) = 1 - (0.36 + 0.16) = 0.48$ | Compare: Entropy = 0.971 (different scale, same conclusion!)

In practice, Entropy and Gini produce very similar trees. sklearn uses Gini by default (criterion='gini').

How Splits Work: The CART Algorithm

sklearn uses CART (Classification and Regression Trees), not ID3. Key difference: CART always makes binary splits (two branches), even for features with many categories.

ID3 can create multi-way splits (one branch per category) — CART always splits into exactly two groups.

Continuous Features

1. Sort all values of the feature
2. Test every midpoint between consecutive values
3. Pick the threshold with best IG (or Gini Gain)

Example: Height values [3.2, 4.8, 5.1, 7.0]
Test thresholds: 4.0, 4.95, 6.05

Categorical Features

CART: Binary split into two subsets
Test all possible 2-way partitions

Example: Color $\in$ {Red, Blue, Green}
Test: {Red} vs {Blue, Green}
Test: {Blue} vs {Red, Green}
Test: {Green} vs {Red, Blue}

For n continuous values, there are $n-1$ candidate thresholds. For k categories, there are $2^k - 1$ possible binary partitions.

Decision Trees for Regression

How It Differs

Instead of class labels, leaf nodes predict a continuous value — the mean of training samples in that leaf. Split criterion: minimize variance (MSE) instead of entropy/Gini.

Split Criterion: MSE

$$\text{MSE} = (1/n) \sum (y_i - \bar{y})^2$$

Minimize weighted average MSE of children after each split. Same IG logic, just with variance.

Example: House Price Prediction

Root: Size $\leq 100\text{m}^2$? $\rightarrow$ Yes: Rooms ≤ 2 ? $\rightarrow$ Yes: €180k |
No: €250k
 $\rightarrow$ No: €420k

**Predictions are
step functions!**

Piecewise constant

sklearn: `DecisionTreeRegressor(criterion='squared_error')` — same tree logic, different split metric.



The Overfitting Problem

Without constraints, a Decision Tree keeps splitting until every leaf is pure — memorizing training data (including noise). Result: perfect training accuracy, terrible test accuracy. Plus: small data changes create completely different trees!

Unconstrained Tree

- Splits until every leaf is pure
- Deep, complex structure
- Train accuracy: ~100%
- **Test accuracy: POOR!**

Pruned Tree

- Limited depth/leaves
- Simpler, generalizes better
- Train accuracy: Good
- **Test accuracy: Good!**

This instability is exactly why ensemble methods (Session 7) were invented — Random Forests use MANY trees to average out variance!

Pruning: Controlling Tree Growth

Pre-Pruning (Early Stopping)

Stop growing BEFORE too complex.

max_depth	Maximum tree depth (e.g., 5)
min_samples_split	Min samples to split (e.g., 10)
min_samples_leaf	Min samples per leaf (e.g., 5)
max_leaf_nodes	Max total leaves (e.g., 20)
max_features	Max features per split

Post-Pruning (CCP)

Grow full tree, then remove unhelpful branches.

Cost Complexity Pruning adds a penalty:

$$\text{Cost} = \text{Error}(T) + \alpha \times |\text{Leaves}|$$

- $\alpha = 0 \rightarrow$ Full tree (no pruning)
- α large $\rightarrow$ Tiny tree (heavy pruning)
- Use cross-validation to find optimal α
- sklearn: `ccp_alpha` parameter

Start with `max_depth=5-10` and `min_samples_leaf=5-10`. These handle most overfitting problems.

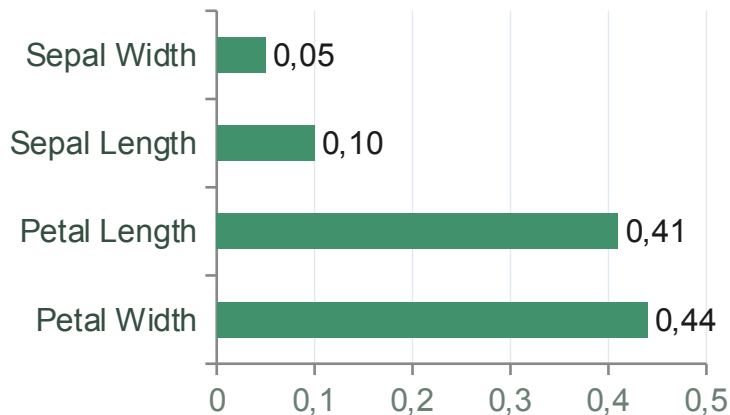
Feature Importance: Which Features Matter?

How It Works

Feature importance measures how much each feature reduces impurity across ALL splits in the tree.

- Weighted impurity decrease summed for each feature
- All importances sum to 1.0
- Higher = more important for predictions
- Unused features → importance = 0

Example: Iris Dataset



```
importances = model.feature_importances_  
pd.Series(importances, index=feature_names).sort_values(ascending=False).plot.barh()
```



Built-in importance can be biased toward high-cardinality features. Use `permutation_importance()` for robustness.

Decision Trees: Advantages & Limitations

✓ Advantages

- ✓ Highly interpretable & visualizable
- ✓ No feature scaling needed
- ✓ Handles numerical + categorical data
- ✓ Captures non-linear relationships
- ✓ Fast training and prediction
- ✓ Built-in feature importance

✗ Limitations

- ✗ Prone to overfitting without pruning
- ✗ Unstable (small data change → different tree)
- ✗ Only axis-aligned splits (no diagonals)
- ✗ Greedy: locally optimal ≠ globally optimal
- ✗ Biased feature importance
- ✗ Single tree often weaker than ensembles

The instability & weakness of single trees is WHY ensemble methods exist — Session 7 covers Random Forests & Boosting!

When to Use Decision Trees

Explainability Required

Medical, credit, regulatory (EU AI Act) — must explain WHY.

Quick Baseline Model

Fast, interpretable first model before trying complex algorithms.

Mixed Feature Types

Numerical + categorical without encoding or scaling.

Feature Selection Tool

Use `feature_importances_` to identify key features for other models.

Avoid for: very high-dimensional data, smooth boundaries needed, or when max accuracy is the only goal.

sklearn: DecisionTreeClassifier

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Create and train the model
tree = DecisionTreeClassifier(
    criterion='gini',          # or 'entropy'
    max_depth=5,              # KEY: prevent overfitting!
    min_samples_leaf=5,       # min samples per leaf
    random_state=42
)
tree.fit(X_train, y_train)

# Evaluate
y_pred = tree.predict(X_test)
print(classification_report(y_test, y_pred))
```

Always set max_depth and min_samples_leaf to prevent overfitting! Start with max_depth=5 and tune from there.

Visualizing Decision Trees in Python

Method 1: plot_tree (Quick)

```
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(15,10))
plot_tree(tree, filled=True,
          feature_names=cols,
          class_names=targets, ax=ax)
```

Method 2: Graphviz (Pretty)

```
from sklearn.tree import export_graphviz
import graphviz

dot = export_graphviz(tree,
                      feature_names=cols,
                      class_names=targets,
                      filled=True, rounded=True)
```

Method 3: Text Export (Debugging)

```
from sklearn.tree import export_text; print(export_text(tree, feature_names=cols))
```

Hyperparameter Tuning with GridSearchCV

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    'max_depth': [3, 5, 7, 10, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 3, 5, 10],
    'criterion': ['gini', 'entropy'],
}

grid = GridSearchCV(DecisionTreeClassifier(random_state=42),
                    param_grid, cv=5, scoring='f1', n_jobs=-1)
grid.fit(X_train, y_train)
print(f'Best: {grid.best_params_} Score: {grid.best_score_:.3f}')
```

Recommended Starting Ranges

Small data: max_depth 3-7, min_samples_leaf 5-10 | Large data: max_depth 5-15, min_samples_leaf 10-50

Use cross-validation (cv=5) to avoid overfitting hyperparameters. Choose scoring metric based on your problem (Session 5!).

Cost Complexity Pruning: Finding Optimal α

```
# Step 1: Get the cost complexity pruning path
path = tree.cost_complexity_pruning_path(X_train, y_train)
alphas = path.ccp_alphas

# Step 2: Train a tree for each alpha value
trees = [DecisionTreeClassifier(ccp_alpha=a, random_state=42).fit(X_train, y_train)
          for a in alphas]

# Step 3: Plot train vs test accuracy for each alpha
train_scores = [t.score(X_train, y_train) for t in trees]
test_scores = [t.score(X_test, y_test) for t in trees]

# Step 4: Choose alpha where test score is highest
optimal_alpha = alphas[np.argmax(test_scores)]
```

Low $\alpha \rightarrow$ Complex tree (overfit) $\rightarrow$ Optimal $\alpha \rightarrow$ Best generalization $\rightarrow$ High $\alpha \rightarrow$ Too simple (underfit)

Plot accuracy vs alpha to visually find the sweet spot — where train and test curves are closest and test is highest.

Real-World Applications

Medical Diagnosis

Symptom-based protocols. "If fever > 38°C AND cough → Test for flu." Interpretability required by law.

Credit Scoring

Approve/deny loans. EU AI Act requires explainable models — trees naturally comply.

Customer Churn

"If usage < 5h/month AND support_tickets > 3 → likely to churn." Actionable rules!

Manufacturing QA

Defect detection from sensor readings. Operators need to understand the rules.



Common Mistakes with Decision Trees

1

No depth limit

→ Always set `max_depth` (start with 5)

2

Accuracy for imbalanced data

→ Use F1, MCC, balanced accuracy (Session 5!)

3

Never visualizing the tree

→ Always inspect with `plot_tree` or `export_text`

4

Trusting feature importance blindly

→ Use permutation importance for robustness

5

Scaling data unnecessarily

→ Trees DON'T need scaling! (But other models do)

6

Expecting smooth predictions

→ Trees produce step functions — that's by design

The #1 mistake: not constraining the tree. An unconstrained tree will ALWAYS overfit.

Hyperparameters Quick Reference

Parameter	Default	Effect	Typical Range
criterion	'gini'	Split quality ('gini' or 'entropy')	Try both
max_depth	None	Max tree depth. KEY overfitting control!	3-15
min_samples_split	2	Min samples to allow a split	2-20
min_samples_leaf	1	Min samples in each leaf	1-20
max_leaf_nodes	None	Maximum total leaves	10-50
max_features	None	Max features considered per split	'sqrt', 'log2'
ccp_alpha	0.0	Cost complexity pruning parameter	0.0-0.05
class_weight	None	Handle imbalanced classes	'balanced'

Start simple: set max_depth=5 and min_samples_leaf=5. Then use GridSearchCV to fine-tune.

Decision Trees vs Previous Models

Aspect	Decision Tree	Logistic Regression	Linear Regression
Type	Both class. & regr.	Classification	Regression
Decision boundary	Non-linear (steps)	Linear	Linear
Interpretability	Very high (visual)	High (coefficients)	High (coefficients)
Feature scaling	Not needed ✓	Required	Recommended
Handles non-linearity	Naturally ✓	Need poly features	Need poly features
Overfitting risk	HIGH (needs pruning)	Low (regularized)	Low (regularized)
Stability	LOW (high variance)	High	High
Missing values	Some support	Must impute	Must impute

No single model is best for everything. Decision Trees shine when interpretability matters and data has non-linear patterns.

Decision Framework: Choosing Your Model

Ask Yourself These Questions

Do you need to explain predictions?

Yes → Decision Tree is a great choice

Is your data imbalanced?

Use `class_weight='balanced'` + MCC/F1 metrics

Are features a mix of types?

Trees handle this natively — no encoding needed

Do you need maximum accuracy?

Consider ensembles (Session 7) or neural nets (Session 9)

Is training speed critical?

Trees are very fast — good for prototyping

Will the model be audited/regulated?

Trees are EU AI Act friendly — use them!

Remember: Decision Trees are a FOUNDATION. Next session, we build ensembles (Random Forest, XGBoost) on top of them!

Session 6 Summary

Key Concepts

- Decision Trees split data using nested rules
- Entropy measures node impurity ($H = -\sum p_i \log_2 p_i$)
- Information Gain = entropy reduction from split
- Gini impurity = alternative (sklearn default)
- Pruning prevents overfitting (max_depth!)
- Feature importance ranks variable usefulness
- Works for classification AND regression
- Highly interpretable — EU AI Act friendly

Key Limitations to Remember

- Always constrain (max_depth, min_samples_leaf)
- Single trees are unstable (high variance)
- Only axis-aligned splits (rectangular regions)
- Greedy algorithm — no global optimum
- Feature importance can be misleading

**These limitations motivate Session 7:
Ensemble Methods (Random Forest, Boosting)**

What's Next?

Session 7: Ensemble Methods

- Random Forests — Many trees > one tree
- Bagging — Bootstrap aggregating to reduce variance
- Boosting — Sequential learning from mistakes
- XGBoost & LightGBM — State-of-the-art for tabular data

"The instability of single trees is not a bug — it's the foundation of Random Forests!"